PRESENTER'S GUIDE

Protocol Lab: live demo

Savir Ghai · 24BCT0309 · Team F1_Team34 · BCSE308P Computer Networks Lab · Prof. Sreethar S

The 8-minute plan

Time	Segment	What the audience should take away
0:00 – 0:30	Open: the problem	Protocols are taught as pictures; this tool lets you change conditions and watch.
0:30 – 1:45	Demo 1: Packet journey	Each layer adds a header; routers rewrite MACs and cut TTL; packets beat circuits here.
1:45 – 3:15	Demo 2: Flow control	Go-Back-N repeats a whole window; Selective Repeat repeats only the lost frame.
3:15 – 5:00	Demo 3: Routing	Plain distance vector counts to infinity; split horizon stops it.
5:00 – 6:15	Demo 4: TCP congestion	Reno halves its window on 3 duplicate ACKs; Tahoe restarts from 1.
6:15 – 7:30	Demo 5: Queuing and QoS	FIFO ruins a phone call; priority and WFQ protect it.
7:30 – 8:00	Close	78 tests, reproducible results, Unit 7 is the next lab. Questions.

Golden rule: open each lab fresh from the top menu (or press F5 on it) so it starts at its defaults, because every number in this guide is the default value. If you change a slider, press F5 and the lab resets.

Before you start (2 minutes)

- ☐ Run `git pull`, then open `f1_team34_index.html` in your browser. It works offline.
- ☐ Light theme for a projector (the button is at the top right); dark theme if the room is dim.
- ☐ Browser zoom at 110 to 125% so the back row can read the numbers (Ctrl and +). Close other tabs.
- ☐ Terminal open in the repo folder with `node tests/f1_team34_tests.js` typed and ready (do not press Enter yet).
- ☐ Slides 9 to 12 of the deck open in another window as a backup if the browser misbehaves.

Opening (30 seconds)

"Networking courses teach protocols with diagrams. A diagram shows one case. I built Protocol Lab so you can change the conditions, break something, and watch what the protocol does. It is eight labs in one HTML page, no install, and the results are tested. I will show five of them."

Numbers to remember

Topic	Default lab (on screen)	30-seed runs (report)
Flow control	GBN repeats 4, SR repeats 1, finish 20.1; Stop-and-Wait 55.1	At 20% loss efficiency 0.405 to 0.681 (GBN to SR)
Routing	Plain: Round 32; split horizon: Round 3	Plain 31 rounds; split horizon 2 to 7 on chains of 3 to 8
TCP	Tahoe 245, Reno 304 (+24%)	Reno +8% to +27%
Queuing	FIFO 160.8 ms and 6.60% loss; priority 5.64 ms; WFQ 7.18 ms; both 0% loss	At 95% load voice p95: 374 ms (FIFO) to 16 ms (WFQ); loss 8.57% to 0%

Live demo

Demo 1: Packet journey (1:15)

One-line message: every layer wraps the data in a header, and routers rewrite the frame but not the IP packet.

#	Do this	You should see	Say
1	Open **Packet journey**. Click **Next step** once.	Step 1 of 7: Host A, application data, 24 B (`GET /index.html HTTP/1.1`).	*This is just text. The application layer does not care how it travels.*
2	Keep clicking **Next step** to step 7.	TCP, then IP, then Ethernet headers are added; packet crosses R1 and R2; MAC addresses change at each router, IP addresses stay.	*Headers are added going down and stripped coming up. A router rebuilds the frame with new MACs. The IP addresses never change.*
3	Click **Restart**. Set **Starting TTL** to 2, then **Next step** to the end.	The packet is dropped at R2 (TTL reached 0).	*This is how traceroute works: it sends packets with a small TTL and listens for the error.*
4	Click **Circuit or packet?** (leave the defaults).	Circuit switching **188 ms**, packet switching **93.7 ms**. Users supported: circuit 10 vs packet 35 (overload chance 0.04%).	*Packets win here: no set-up and the links work in parallel. Circuits win when set-up is cheap and a steady rate is needed, like a phone call.*

Demo 2: Flow control (1:30)

One-line message: when a frame is lost, Go-Back-N repeats a whole window, Selective Repeat repeats only that frame.

#	Do this	You should see	Say
1	Open **Flow control**. Click **Go-Back-N**, then **Show everything**.	Defaults: 10 frames, window 4, frame 2 always lost. Message: “receiver dropped 3 frames and the sender repeated 4”. Finished at **20.1**.	*One frame was lost, but the receiver cannot keep frames that arrive after the gap, so the sender resends the whole window.*
2	Click **Selective Repeat**, then **Show everything**.	“1 repeat for 1 timeout”. Finished at **20.1** as well.	*Selective Repeat buffers the early frames and resends only frame 2. Same finish time here, much less waste.*
3	Click **Stop-and-Wait**, then **Show everything**.	Finished at **55.1**.	*Stop-and-Wait sends one frame per round trip. This is why we need a window.*
4	Click **Efficiency vs frame loss**.	The Selective Repeat line stays above Go-Back-N as loss rises.	*In my 30-seed runs at 20% loss, efficiency goes from 0.405 to 0.681 and retransmissions per frame from 1.46 to 0.25.*

Demo 3: Routing (1:45)

One-line message: distance vector can count to infinity after a link breaks; split horizon stops it.

#	Do this	You should see	Say
1	Open **Routing**, **Link state**, six-router network. Click **Next step** until it ends.	Dijkstra from A finalises B (7), C (9), F (11), then D and E (20).	*Link state: every router knows the whole map and runs Dijkstra. One router is finalised each step.*

#	Do this	You should see	Say
2	Click **Distance vector**. Network: **Three in a line**. Fix: **None**. Click **Run until stable**.	"Round 1 (stable)".	*Three routers, A, B and C. They have learned each other's routes.*
3	Select link **B – C**, click **Break link**, then **Run until stable**.	The chart climbs until cost 32 (infinity). The screen ends at **Round 32 (stable)**.	*A and B keep telling each other they can reach C through the other, so the cost creeps up by one each round. That is count to infinity.*
4	Set fix to **Split horizon**. Click **Run until stable**, select **B – C**, **Break link**, **Run until stable**.	Ends at **Round 3 (stable)**.	*Split horizon: do not advertise a route back to the router you learned it from. The loop never starts.*

If someone asks "you said 31 rounds in the report, the screen says 32": 31 rounds change something, and the 32nd round confirms that nothing changed. Same for split horizon: 2 changing rounds plus 1 to confirm, so the screen shows 3.

Be honest about the limit: split horizon only stops loops between two neighbours. It does not fix loops in a larger mesh, and this demo does not test that.

Demo 4: TCP congestion (1:15)

One-line message: on three duplicate ACKs Reno halves its window and Tahoe restarts from 1.

#	Do this	You should see	Say
1	Open **TCP congestion**. Stay on **Window lab: Tahoe and Reno** (default).	Two lines over 26 rounds: a triple duplicate ACK at round 9 and a timeout at round 17.	*The window doubles in slow start, then grows by one per round in congestion avoidance.*
2	Point at round 9, then round 17.	At round 9 Tahoe drops to 1 and Reno drops to half. At round 17 both drop to 1.	*Three duplicate ACKs mean the network is still delivering something. Reno takes the mild reaction; a timeout is serious for both.*
3	Point at the readout under the chart.	Tahoe **245** segments, Reno **304**, Reno about **24%** ahead.	*Same losses for both. Reno sends 24% more because it recovers faster.*

Caveat to say yourself: in my 30-seed runs Reno is ahead by 8% to 27%, but the model draws one loss event per round whatever the window size, so the size of the gain depends on the model. If asked for more, the **Shared link** tab shows flows sharing a router with drop-tail and RED.

Demo 5: Queuing and QoS (1:15)

One-line message: the queuing rule decides whether a phone call survives a busy link.

#	Do this	You should see	Say
1	Open **Queuing and QoS** (FIFO is selected by default).	Voice delay spikes up to about 400 ms. Voice: mean delay **160.8 ms**, loss **6.60%**. Red note: the call would sound poor. Call quality: **Poor: delay, loss**.	*Each point is a voice packet. The spikes are voice packets stuck behind big 1500-byte data packets in a first-in-first-out queue.*
2	Click **Priority queuing**.	The line goes flat. Voice mean delay **5.64 ms**, loss **0.00%**. Call quality: **Good**.	*Voice is always served first.*
3	Click **Weighted fair queuing**.	Voice mean delay **7.18 ms**, loss **0.00%**. Call quality: **Good**.	*Slightly slower than priority for voice, but each flow gets a guaranteed share, so bulk and web traffic cannot be starved. That is why it is preferred in practice.*

A good call means: 95th percentile delay under 150 ms, jitter under 30 ms and loss under 1%. My 30-seed runs at 95% data load: voice p95 delay 374 ms with FIFO against 16 ms with WFQ, and loss 8.57% against 0%.

Close (30 seconds)

"Eight labs, four baseline-against-improvement comparisons measured over 30 seeds, and 78 tests. Let me show the tests." Press Enter in the terminal: ***78 passed, 0 failed****. "The one gap is Unit 7: there is no DNS lab yet, and that is my next build. Questions?"*

If asked: the other three labs

Each of these takes 30 to 45 seconds. Use them only if someone asks, or if you finish early.

Lab	Do this	You should see
Error detection (Hamming)	Open the lab, Hamming code tab (default data 1011001). Click **Flip 1 random bit**, then **Flip 2 random bits**.	7 data bits need 4 parity bits: an 11-bit codeword, code rate 0.64. One flipped bit is located and corrected. Two flipped bits are mis-corrected by plain Hamming (the extended option detects them).
Multiple access	Open the lab and click **Compare all protocols**.	Throughput against load: pure ALOHA peaks at about 0.184, slotted ALOHA at about 0.368; CSMA does better when the wire is short (small a).
Subnetting	Calculator: type 192.168.10.37 with /26. Then **Divide a network**: 192.168.10.0/24 with hosts 100, 50, 20, 2.	/26: network 192.168.10.0, broadcast 192.168.10.63, 62 usable hosts. VLSM gives /25, /26, /27, /30 and uses 228 of 256 addresses.

If something goes wrong

- **A lab looks stuck or shows odd values:** press F5. The lab resets to its defaults.
- **Browser misbehaves or no projector for the browser:** switch to the deck, slides 9 to 12 show the same screens and numbers.
- **Numbers differ from this guide:** you changed a slider. Press F5. The numbers here are the defaults.
- **Text too small:** Ctrl and + in the browser. **Room too bright:** use the light theme.
- **node not found:** skip the live test run and say "78 passed, 0 failed" is in the report, Appendix B.
- **Offline:** nothing needs the internet. Fonts fall back to system fonts.

Likely questions and short answers

Question	Answer
What is actually new here?	Not a new protocol. The combination: eight labs in one zero-install page, baseline and improvement side by side on identical input, tested against known answers, and reproducible from a seed.
How do you know the simulations are right?	78 automated tests. Four check known results: CRC-32 of 123456789 is 0xCBF43926, the IPv4 checksum example is 0xB861, pure ALOHA peaks at 0.184, and Dijkstra agrees with distance vector. The rest check invariants like exactly-once, in-order delivery under loss.
Why is infinity 32 and not 16 like RIP?	The example network has a legitimate path cost of 20, which 16 would clip. It is documented in the lab.
The report says 31 rounds but the screen says 32?	31 rounds change something; the next round confirms nothing changed, so the screen shows 32. Split horizon: 2 changing rounds plus 1 confirming, shown as 3.
Does split horizon fix all routing loops?	No. It stops loops between two neighbours. Larger loops in a mesh need other measures such as triggered updates or path information. I tested chains only.
Why does Selective Repeat win, and what does it cost?	It resends only lost frames: at 20% loss, 0.25 retransmissions per frame against 1.46 for Go-Back-N. The cost is a receiver buffer and per-frame acknowledgements.
Why is the Reno gain model-dependent?	The TCP lab draws one loss event per round, regardless of window size. In a real network a bigger window sees more losses. So the direction is right but do not quote the percentage as real-world.
Why WFQ over priority queuing?	Priority is faster for voice (5.64 ms against 7.18 ms) but can starve other traffic if voice load is high. WFQ guarantees every flow a share, at the cost of a few milliseconds.
Why 30 seeds, and is it statistically proven?	Thirty runs give a stable mean and standard deviation. I did not run significance tests; the claim is that the means differ by many standard deviations in the medium and high stress scenarios.
Why not ns-3 or Packet Tracer?	ns-3 is more accurate but needs installation and scripting. Packet Tracer was not usable here and hides protocol internals. This tool is for fast, visible, repeatable teaching experiments, not research accuracy.
What does it not cover?	Unit 7 (DNS, HTTP, FTP) has no lab; 802.11 CSMA/CA is theory only; TCP and queuing are simplified models; no user study was done.
Why do packets beat circuits in your example?	A 100 ms circuit set-up against 100 packets of 1000 B pipelined over 4 links: 188 ms against 93.7 ms. Circuits win when the set-up is cheap, the message is large and a steady guaranteed rate matters.
How do I reproduce your results?	`node scripts/f1_team34_experiments.js` reruns all four comparisons in about half a second. `node tests/f1_team34_tests.js` runs the 78 tests.